TFlite delegate と rv32emc を RTL で実装

2023/03/04　　s.yamashita

課題

- 車両前方の画像および全方位の点群データから、3D 物体検出を行うアルゴリズムを作成
- 開発したアルゴリズムを、RISC-V を搭載したプラットフォームに実装

概要

- 今回の課題に対し、TensorFlow Lite(以下 TFlite) の delegate 機構を用いた FPGA アクセラレータを開発した
  (第 5 回コンテストで用いたものを out channel 4 並列実行するよう修正して用いた)

- RISC-V は rv32emc に対応する CPU core を scratch から開発した
  (第 5 回コンテストで用いたものに fpu を組み込んだ。fpu は整数レジスタを用いる形式 “Zfinx” とした)

- アクセラレータの実行制御と、LiDAR data から BEV 画像を生成する pre 処理を 1 つの RISC-V で行った

- FPGA への実装は、アクセラレータ、RISC-V core とも SystemVerilog を用いた RTL 記述で行った

- 推論ネットワークは Super-Fast-Accurate-3D-Object-Detection-PyTorch[*1] をベースに学習を行い、pytorch → TFlite 変換の後 8bit 量子化を行った

- アプリケーションは TFlite の python インターフェースと python C API を用い、RISC-V での BEV 画像生成処理は C 言語で開発した

推論ネットワーク

- 3D 物体検出のベースネットワークは Super-Fast-Accurate-3D-Object-Detection-PyTorch[*2] (SFA3D) を選択した
  車両全方位の LiDAR データから BEV 鳥瞰画像を生成し、fpn_resnet_18 を用いた centernet で物

[*1] https://github.com/maudzung/SFA3D [^1 SFA3D](https://github.com/maudzung/SFA3D)
[*2] https://github.com/maudzung/SFA3D [^1 SFA3D](https://github.com/maudzung/SFA3D)

体中心座標を推論する

- 学習には提供された OperaDataset のうち、全方位でアノテーションされているデータのみ用い、train 90% val 10% に分けて学習した

- 今回の課題では物体の中心座標のみ求めるので、動作速度の観点でネットワークの出力を heatmap と center offset に絞った
  また、fpn_resnet_18 の中の ReLU を ReLU6 に変更したところ量子化後の精度が若干改善した

- 学習後の pytorch ネットワークを TFlite に変換し、8bit 量子化を行った
  - pytorch model を openvino2tensorflow[*3] で TFlite に変換し、Training 後の量子化を行って、8bit に量子化した

  - TFlite のネットワークを編集し、Pad+Conv2D(valid) を Conv2D(same) に置き換えた (図 1)
    pytorch → TFlite 変換で、Conv2D の仕様の不整合[*4]から pytorch:Conv2D(same) -> TFlite:Pad+Conv2D(valid) に変換されてしまう (余分な Pad 処理が発生する)
    Conv2D の stride 設定が 1 のときは単純に Pad をバイパスし、valid を same に書き換えればよいが、stride が 2 のときに Conv2D の出力位相が合わなくなる。stride が 2 のノードは FPGA のアクセラレータの設定を修正して位相を合わせるようにした

図 1. pytorch → TFlite 変換で余分な PAD が発生する問題と対処

- ベースのネットワークは BEV 画像サイズが 608x608 であるが、FPGA での実行時間と推論精度のトレードオフを取るため 320x320 及び 448x448 のネットワークも学習した
  図 2 に BEV 画像と heatmap 画像の例 (320x320) を示す
  【国立大学法人東海国立大学機構名古屋大学・一般社団法人人間機械協奏技術コンソーシアムのデータセットを利用】

- BEV の幅は、100m 相当にしたが、幅 84m 相当にして車体進行方向を 45 度傾けたパターン (R45 図 2 下) も試した
  幅 84m では車の検出範囲 50m を超える領域が発生し車の検出を取りこぼすが、検出しにくい歩行者の検出率を上げる効果がある
  また、車体斜め 45 度方向の車の出現頻度は車体正面より少ない様であり、実用的にも車体正面が重要であるので正面の検出範囲を広げるのが良いと考えた

図 2.BEV 画像と heatmap 画像

- 図 3 は BEV の画像サイズ (S/M/L : 320/448/608) と、45 度傾けたパターン (R45) による Average

[*3] https://qiita.com/PINTO/items/7a0bcaacc77bb5d6abb1 [^2 openvino2tensorflow](https://qiita.com/PINTO/items/7a0bcaacc77bb5d6abb1)

[*4] https://stackoverflow.com/questions/52975843/comparing-conv2d-with-padding-between-tensorflow-and-pytorch [^3 comparing-conv2d-with-padding-between-tensorflow-and-pytorch](https://stackoverflow.com/questions/52975843/comparing-conv2d-with-padding-between-tensorflow-and-pytorch)

Precision の変化である
計測は、KV-260 上で実行した。用いたデータは学習時に val 用に分けたデータである
train と val のデータは似通っているので AP が高めに出ている。(リーダーボードではサイズ 608 で mAP = 0.32 であり大きな乖離がある)
また、FPGA で実行した AP は CPU で実行した AP より若干 (~4% 程度) 低い。整数演算であるが、Acc の丸め処理に若干差がありこれが要因の可能性がある

- BEV 画像サイズが小さくなると、特に歩行者の検出率が顕著に低下する
  R45 では BEV 画像サイズが小さいときの歩行者の検出率が上がっており、効果が認められる

図 3.BEV 画像サイズと R45 による Average Precision の比較

- 以下は、TFlite の benchmark tool を KV-260 の CPU Arm® Cortex-A53 上で実行し、今回のネットワーク (608x608) を分析した結果である。
  Arm での推論実行時間は約 7.55 s であり、Conv2D の演算で 96.2% を占める。Conv2D 演算を FPGA に delegate する

```
Number of nodes executed: 79
============================ Summary by node type ===========
             [Node type] [count] [avg ms]  [avg %]   [cdf %]
                 CONV_2D      35 7264.019  96.151%   96.151%
        RESIZE_BILINEAR       3  121.071   1.603%   97.754%
                    ADD       8   45.425   0.601%   98.355%
                    SUM       2   44.592   0.590%   98.945%
               QUANTIZE      11   25.896   0.343%   99.288%
          CONCATENATION       5   22.259   0.295%   99.583%
            MAX_POOL_2D       1   15.889   0.210%   99.793%
                SOFTMAX       2   11.255   0.149%   99.942%
RESIZE_NEAREST_NEIGHBOR       2    2.091   0.028%   99.969%
                    MUL       2    1.682   0.022%   99.992%
             DEQUANTIZE       2    0.441   0.006%   99.998%
                RESHAPE       6    0.182   0.002%  100.000%
                          total 7554.84 ms
```

全体システム構成

図 4 にシステム構成を示す。
TFlite の Interpreter（推論プログラム）には外部のアクセラレータを接続する delegate API があり[*5]、これ

[*5] https://www.tensorflow.org/lite/performance/implementing_delegate [^4 implementing_delegate](https://www.tensorflow.org/lite/performance/implementing_delegate)

を用いて FPGA に演算を委譲する。

図 4. システム構成

新たに作成した FPGA とのインターフェース関数 dummy_external_delegate.so を Interpreter にリンクする。
インターフェース関数は起動時に delegate する演算種別 Conv2D, depthwiseConv2D の 2 つを登録する。(今回のネットワークでは depthwise は使われていないが、回路規模は変わらないので残してある)
Interpreter を起動すると、FlatBuffer 形式の graph を実行し、登録した演算のみインターフェース関数に渡される。
インターフェース関数には Conv 演算のパラメータと、input, filter, bias, output の 4 つの Tensor へのポインターが渡されるので、パラメータとポインターを FPGA に渡してハードウェアのシーケンサーを kick し、演算終了を待つ。
Tensor へのポインターは、Linux の仮想記憶領域にあり、直接 FPGA からアクセスできない。
Interpreter 起動時、インターフェース関数の Prepare() が一度だけ呼ばれるので、この中で予め Tensor データを CMA 領域に copy しておき、この領域で FPGA とデータをやり取りする。
また、output channel ごとの量子化パラメータ quant を計算する必要があり、一度だけ計算して CMA 領域に保持する。
今回は、第 5 回コンテストで用いたアクセラレータに、output channel を 4 並列で処理するように変更を行ったが、アクセラレータの Conv filter 係数のアクセス順序が連続するように filter 係数の並べ替えを行った。この並べかえも Prepare() で行った。

FPGA に渡した Conv 演算パラメータは、FPGA 側に設けた RISC-V CPU で並列演算に対応した形に変換して Conv 演算アクセラレータのシーケンサに与え、kick する。
RISC-V は、シーケンサの終了を待って、output buffer に残ったデータを CMA 領域に flush して演算を終了し、アプリケーションに通知する。

以上が Interpreter と FPGA アクセラレーターが連携して推論ネットワークの演算を行う流れである。
今回の課題では、LiDAR data を BEV 画像に変換する pre 処理、ネットワークの出力である heatmap から物体位置を計算する post 処理が必要である。
これらの処理のうち、より多くの演算処理の必要な pre 処理を、RISC-V で実行した。RISC-V のクロックは 100MHz と遅く実行時間がかかるが、Interpreter の推論時間を下回ることができた。このため pre 処理と Interpreter の推論処理を並行して実行するようにして pre 処理の実行時間を隠蔽することができた。

アクセラレーターの制御は RISC-V の割り込み処理で行っており、pre 処理と平行に実行される。

rv32emc core のハードウェア

第 5 回コンテストで用いた RISC-V CPU は、SystemVerilog で scratch から記述したものである。
RISC-V の ISA のうち、組み込み用途向けの EMC (32bit 16 Register, Mul/Div, Compressed 命令) の構成

で開発した。[*6]
今回この CPU に fpu を追加した。
fp レジスタを設けずに、整数レジスタを共用する Zfinx 仕様[*7]としてハード規模を抑えた。ただし、fmadd/fmsub 命令は 3 オペランド命令で追加による変更規模が大きく、実装しなかった。
クロスコンパイラは riscv-gnu-toolchain で Zfinx に対応したもの[*8] (gcc バージョンは 10.2.0) に改造を加え fmadd/fmsub を使わないようにする修正を行っている。

[^7] にしたがってインストール
`zfinx/riscv-gcc/gcc/config/riscv/riscv.md` を修正→ `fmadd/fmsub` を生成する条件を `disable`

```
$ ./configure --prefix=/opt/rv32e/ --with-abi=ilp32e --with-arch=rv32emczfinx --with-multilib-generator='rv32emczfinx-ilp32e--'
$ make newlib
```

FPGA に実装したハードウエア構成

図 5 に FPGA PL 部に実装したアクセラレータのブロック構成を示す。
第 5 回コンテストで用いた回路を output channel 4 並列処理できるように修正し、制御及び LiDAR data プレ処理用 CPU として前述の RISC-V CPU を用いた。
また、RISC-V とアクセラレータのクロックを分離し、コントロールが非同期でできるように改修したので、RISC-V とアクセラレータのクロック周波数を独立して設定できるようになった。

PS のアプリケーションは、AXI-APB bridge を介して RISC-V の RAM をアクセスすることができる。
RISC-V の実行バイナリを RAM にロードし、reset を解除することで RISC-V を起動する。
また、APB から 割り込みを発生し、アクセラレータの制御を kick する。

図 5. PL ブロック図

Accelerator 部では、Tensor data のアドレスを発生する adrgen からのアドレスにしたがって Tensor data にアクセスする。
畳み込み演算回路 (MAC) で input tensor と filter tensor を乗算、累算し、bias を加えたのち 8bit に量子化し、output tensor に出力する。Conv2D と dwConv2D の 2 種の演算で MAC 回路は共通であり、アドレス発生パターンが異なるのみである。
input, filter, bias, quant, output の 5 つのデータアクセスは、S_AXI_HPC0_FPD ポートを介して PS の CMA 領域をアクセスする。
これらのデータは、output channel 4 並列単位でアクセスされる。

[*6] https://github.com/shin-yamashita/rv32emc [^5 github shin-yamashita/rv32emc](https://github.com/shin-yamashita/rv32emc)
[*7] https://wiki.riscv.org/display/HOME/Zfinx+TG [^6 Zfinx+TG riscv.org](https://wiki.riscv.org/display/HOME/Zfinx+TG)
[*8] https://github.com/pz9115/riscv-gcc/wiki/How-to-regression-test [^7 risc-v gcc for Zfinx (How-to-regression-test)](https://github.com/pz9115/riscv-gcc/wiki/How-to-regression-test)

MAC は、Np(x4) 個並列に実装されており、input, output のデータアクセスには Np 個並列にキャッシュメモリを設けた。filter, bias, quant は全ての MAC 演算で共通なので、それぞれ 1 個のキャッシュメモリを設けた。
filter, bias, quant, output は連続アクセスなので、FIFO バッファのような構造で良いが、input は畳み込みのタップで大きくアドレスが飛ぶので、16 ラインのキャッシュメモリ構造にした。(図 6.input-cache ブロック図 参照)

図 6. input-cache ブロック図

図 7. Np 分割

並列化のアドレス分割は、図 7 に示すように、output Tensor の W x H を Np 分割する。C を共通にすることで、filter, bias, quant のキャッシュが 1 系統ですむ。ただし、W x H < Np のときは並列数は W x H に制限される。

インターフェース関数から渡される演算パラメータは、M_AXI_HPM0_FPD ポート axi-apb ブリッジを介し、アクセラレータ制御用 RISC-V CPU に渡される。
Np 個並列の input/output それぞれに異なるアドレス発生を行うため、CPU は adrgen ブロックに Np 種類のアドレスを計算し、設定を行う。
並列に実装するアドレス発生回路をできるだけ簡単化してハード規模を抑えるために、CPU にアドレス計算を分担するようにした。また、この CPU にはシリアルターミナルを接続できるようにし、デバッグおよび実行時間の観測などにも活用した。

■論理合成、FPGA 実装

作成した RTL module を Vivado の Block Design ツールに読み込んで Zynq と接続した。
(図 8. Block Design　色付きのブロックが RTL ブロック)
Np 、キャッシュサイズ、クロック周波数などパラメータを変えて論理合成を繰り返し、エリア及びタイミングが MET できるように RTL 記述にフィードバックした。

図 8. Block Design

図 9 は最終的な FPGA のリソース使用率である。
　並列数 Np = 32 (MAC 128 並列)
　アクセラレータクロック周波数 = 187 MHz
　RISC-V クロック周波数 = 100MHz
BRAM が 100% 近くになっており、これ以上の並列数増加は難しい。また使用率が上がるに従ってタイミングが厳しくなり、クロック周波数を上げられない。
一方で LUT / DSP にはまだ余裕があり、FPGA リソースを最大限に活用するには構造の見直しが必要である。

図 9. Utilization

アプリケーション

今回、推論の実行アプリケーションは python で作成した。
ライブラリ tflite_runtime[*9] を用いる。(KV260 用に tensorflow のソースツリーで cross build する必要があった)
新たに作成した delegate インターフェースは、ダイナミックリンクライブラリ (dummy_external_delegate.so) として tflite_runtime とリンクすることで実行の delegate が行われる。
tflite_runtime の python インターフェースでは interpreter のインスタンス時に dummy_external_delegate.so を指定してリンクするだけである。

RISC-V と推論実行アプリのシーケンス。

以下、推論実行のシーケンスである

1. LiDAR data を file から読み込み、python C API を介して preproc module に渡す
2. preproc module は LiDAR data を CMA 領域にコピーし、RISC-V を kick する
3. RISC-V は LiDAR data から BEV 画像への変換処理を行う
4. 1 frame 前の preproc で生成された BEV 画像を TFlite interpreter に set_tensor() して invoke() する

[*9] https://www.tensorflow.org/lite/guide/python [^8 tflite_runtime](https://www.tensorflow.org/lite/guide/python)

```
import tflite_runtime.interpreter as tflite
# Instantiate interpreter
interpreter = tflite.Interpreter(model_path=model,
  experimental_delegates=[tflite.load_delegate(
                '{path-to}/dummy_external_delegate.so.1')] )
interpreter.allocate_tensors()
# Get input and output tensors.
input_details  = interpreter.get_input_details()
output_details = interpreter.get_output_details()
  :
# preproc
bev_image = preproc.preproc(frame)  # LiDAR data -> BEV image
# set image
interpreter.set_tensor(input_details[0]['index'], np.uint8(bev_image))
# Invoke
interpreter.invoke()
# Output inference result
heatmap = interpreter.get_tensor(output_details[0]['index'])[0]
cen_offset = interpreter.get_tensor(output_details[1]['index'])[0]
# postproc
det = postproc.postproc(heatmap, cen_offset)
  :
```

5. 推論結果の heatmap と center-offset から物体位置を計算し、json に出力する

処理 3. と処理 4. が並列に行われる。
RISC-V ではアクセラレータの制御は割り込み処理によって行われ、preproc の BEV 画像生成処理と並列に実行される。

図 10. 推論実行アプリシーケンス

実行結果

■推論処理時間詳細

推論時間は、PS の Arm と FPGA/RISC-V で分担されており、FPGA の実行時間は FPGA 内に実装したカウンタで測定した。
図 11 に結果を示す。

図 11. 推論処理時間

グラフの Conv(黄色) 部分、畳み込み演算そのものの実行時間が支配的である。
RISC-V の preproc は並行して処理され、表には現れない。

まとめ、今後の課題

- TensorFlow Lite の delegate 機構を用いた FPGA アクセラレータを開発し、課題に対する動作を確認することができた。
  この方式は、既存の TFlite のフレームワークを変えることなく、そのまま使える特徴がある。
- アクセラレータは output チャンネルの 4 並列化を実装し、MAC の並列数を 128 並列にすることができた。ソフトウェアで filter 係数の並べ替えを行うことでハードウエアの複雑化を回避した。
- RISC-V (rv32emc) を RTL で実装し、FPGA に組み込んで、アクセラレータ制御と LiDAR データの前処理に用いた。Zfinx 仕様の fpu を新たに開発して組み込み、LiDAR data の処理に活用した。

今後の課題

- Arm での推論実行に対し、FPGA での実行で推論精度が若干低下することがわかっている。acc 回路の丸めは Arm の処理より下位ビットを削っており、丸め精度について詳細に調査する必要がある。
- 動作速度は RISC-V 100MHz、アクセラレータ 187MHz が限界であった。クリチカルパスを調査し、論理段数を減らす必要がある。
  アクセラレータは pipeline 化の余地がある。RISC-V は compress 命令対応が速度低下の一因であるので仕様変更が有効かもしれない。

- これまでアクセラレータに delegate する演算を Conv2D に限ってきたが、ハード規模の増大を抑えつつ対応できる演算を増やすことを考えてみたい。